

Simulador de Gestor de Procesos para Sistemas Operativos

Documento técnico universitario

Autores:

Narez Roses Erick Eduardo

Jimenez Aguilar Christian Jesus

Llamas Sanchez Adrian

Meza Roman Juan Pablo

Institución: Universidad Autonoma de Tamaulipas

Fecha: 19/05/2026

Asignatura: Sistemas Operativos

Lenguaje de implementación: Java 17

Gestor de construcción: Maven

Interfaz: Consola interactiva (CLI)

Índice

1. Estructura del Repositorio GitHub	5
1.1. Nombre del proyecto	5
1.2. Descripción breve	5
1.3. Tecnologías utilizadas	5
1.4. Estructura de carpetas Maven	5
1.5. Explicación de carpetas y paquetes	7
1.6. Clases principales	8
1.7. Código fuente del proyecto	8
1.8. Explicación del archivo pom.xml	13
1.9. Instrucciones de compilación y ejecución	14
1.10. Estructura recomendada del README.md	14
2. Introducción	15
3. Objetivo general	15
4. Objetivos específicos	15
5. Planteamiento del problema	15
6. Justificación	16
7. Descripción general del sistema	16
8. Arquitectura del sistema	16
9. Explicación de módulos	17
9.1. main	17
9.2. models	17
9.3. managers	17

9.4. schedulers	17
9.5. synchronization	17
9.6. utils	17
10.Gestión de procesos	17
11.Estados de procesos	18
12.Planificación de CPU	18
13.Explicación de algoritmos	18
13.1. FCFS	18
13.2. SJF	18
13.3. Round Robin	18
13.4. Prioridades	19
14.Administración de recursos	19
15.Comunicación entre procesos	19
16.Sincronización	19
17.Problema productor-consumidor	19
18.Manejo de memoria	20
19.Logs del sistema	20
20.Estadísticas del sistema	20
21.Flujo de ejecución	20
22.Casos de uso	21

23.Ejemplos de ejecución	21
24.Pruebas del sistema	21
24.1. Prueba 1: compilación del proyecto	21
24.2. Prueba 2: ejecución del menú principal	22
24.3. Prueba 3: creación de procesos	22
24.4. Prueba 4: planificación de CPU	23
24.5. Prueba 5: comunicación y sincronización	25
24.6. Prueba 6: logs y estadísticas	26
25.Resultados esperados	27
26.Conclusiones	27
27.Referencias	28
A. Resumen de clases reales del proyecto	29

1. Estructura del Repositorio GitHub

1.1. Nombre del proyecto

Simulador de Gestor de Procesos para Sistemas Operativos.

1.2. Descripción breve

Proyecto desarrollado en Java con Maven y una interfaz de consola interactiva (CLI) que simula el comportamiento básico de un sistema operativo en la administración de procesos, recursos, planificación de CPU, sincronización, comunicación entre procesos, logs y estadísticas.

1.3. Tecnologías utilizadas

- **Lenguaje:** Java 17.
- **Gestor de dependencias y construcción:** Maven.
- **Interfaz:** Consola interactiva (CLI).
- **API de concurrencia:** semáforos y sincronización con Java.
- **Estructura de proyecto:** Maven estándar.

1.4. Estructura de carpetas Maven

La siguiente estructura corresponde a la organización real del repositorio, incluyendo archivos principales, código fuente, documentación y capturas de evidencia.

```
GestorDeProcesos/
|-- .gitignore
|-- pom.xml
|-- README.md
|-- DOCUMENTO_TECNICO_PROYECTO.txt
|-- GUIA_PRACTICA_SIMULADOR.txt
|-- estructura.txt
|-- src/
|   |-- main/
|       |-- java/
|           |-- com/
|               |-- gestorprocesos/
|                   |-- main/
|                       |-- App.java
|                   |-- managers/
|                       |-- ProcessManager.java
|                       |-- ResourceManager.java
|                   |-- models/
|                       |-- EstadoProceso.java
```

```

|         | |-- MensajeProceso.java
|         | |-- MotivoTerminacion.java
|         | |-- Proceso.java
|         | '-- SchedulerType.java
|         |-- schedulers/
|         | |-- AbstractScheduler.java
|         | |-- FcfsScheduler.java
|         | |-- SjfScheduler.java
|         | |-- RoundRobinScheduler.java
|         | |-- PriorityScheduler.java
|         | |-- Scheduler.java
|         | '-- SchedulerFactory.java
|         |-- synchronization/
|         | |-- MessageQueue.java
|         | '-- ProducerConsumerSimulation.java
|         '-- utils/
|         | |-- ConsoleInput.java
|         | |-- Logger.java
|         | |-- RandomDataGenerator.java
|         | |-- StatisticsManager.java
|         '-- TimeSimulator.java
|-- docs/
| '-- entregable_final.pdf
'-- capturas/
    |-- compilacion_del_proyecto.png
    |-- comunicacion_entre_procesos.png
    |-- creacion_de_procesos.png
    |-- ejecucion_del_menu_principal.png
    |-- ejecucion_fcfs.png
    |-- ejecucion_prioridades.png
    |-- ejecucion_sjf.png
    |-- logs_del_sistema.png
    |-- planificacion_de_cpu.png
    |-- round_robin.png
    '-- simulacion_producto_consumidor.png

```

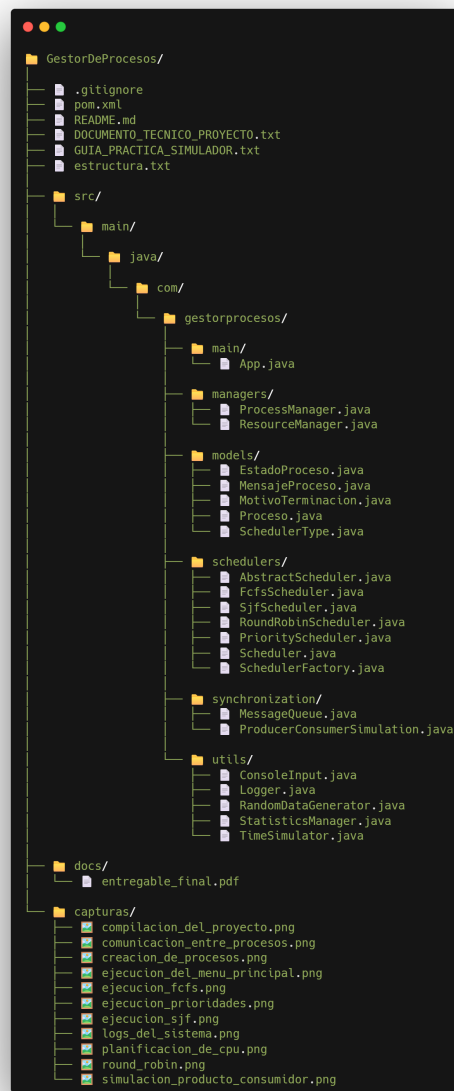


Figura 1: Estructura real de carpetas del repositorio.

1.5. Explicación de carpetas y paquetes

- **/src/main/java/com/gestorprocesos/main:** contiene el punto de entrada de la aplicación. Aquí se ubica `App`, que carga el menú principal y coordina el flujo del simulador.
- **/src/main/java/com/gestorprocesos/models:** contiene las entidades y enums del dominio. Aquí están `Proceso`, `MensajeProceso`, `EstadoProceso`, `MotivoTerminacion` y `SchedulerType`.
- **/src/main/java/com/gestorprocesos/managers:** contiene la lógica de negocio principal para administración de procesos y recursos. Aquí están `ProcessManager` y `ResourceManager`.
- **/src/main/java/com/gestorprocesos/schedulers:** contiene la estrategia de planificación.

Aquí están `Scheduler`, `AbstractScheduler`, `FcfsScheduler`, `SjfScheduler`, `RoundRobinScheduler`, `PriorityScheduler` y `SchedulerFactory`.

- **/src/main/java/com/gestorprocesos/synchronization:** contiene la simulación de sincronización y comunicación entre procesos. Aquí están `MessageQueue` y `ProducerConsumerSimulation`.
- **/src/main/java/com/gestorprocesos/Utils:** contiene clases de apoyo: `Logger`, `ConsoleInput`, `RandomDataGenerator`, `StatisticsManager` y `TimeSimulator`.
- **/docs:** contiene el archivo entregable en PDF del proyecto.
- **/capturas:** almacena evidencias visuales de compilación, ejecución del menú, creación de procesos, planificación, comunicación, productor-consumidor y logs.
- **Archivos raíz:** incluyen la configuración Maven, el README, guías, documento técnico y archivo de estructura.

1.6. Clases principales

Clase	Descripción
App	Clase principal del sistema. Muestra el menú CLI, recibe entradas del usuario y coordina los módulos.
Proceso	Representa la PCB (Process Control Block) del proceso simulado.
ProcessManager	Gestiona la creación, suspensión, reanudación, terminación, mensajería e historial de procesos.
ResourceManager	Administra CPU, memoria y recursos compartidos.
Scheduler	Interfaz base de planificación.
FcfsScheduler, SjfScheduler, RoundRobinScheduler, PriorityScheduler	Implementan los algoritmos de planificación.
ProducerConsumerSimulation	Simula el problema productor-consumidor con semáforos.
Logger	Registra eventos del sistema en consola y en memoria.
StatisticsManager	Calcula estadísticas de ejecución y memoria.
RandomDataGenerator	Genera procesos de prueba de forma aleatoria.

1.7. Código fuente del proyecto

En esta parte se muestran capturas del código fuente de las clases principales disponibles. Cada imagen se acompaña con una explicación breve de la función que cumple dentro del simulador.

Scheduler.java

Define la interfaz base de planificación. Sirve como contrato para que todos los algoritmos implementen una forma común de seleccionar y ejecutar procesos.


```

1 package com.gestorprocesos.schedulers;
2
3 import java.util.List;
4
5 import com.gestorprocesos.managers.ProcessManager;
6 import com.gestorprocesos.managers.ResourceManager;
7 import com.gestorprocesos.models.Proceso;
8 import com.gestorprocesos.utils.Logger;
9 import com.gestorprocesos.utils.StatisticsManager;
10
11 public interface Scheduler {
12     String getNombre();
13
14     void agregarProceso(Proceso proceso);
15
16     Proceso obtenerSiguienteProceso();
17
18     void ejecutar(ProcessManager processManager, ResourceManager resourceManager, StatisticsManager statisticsManager, Logger logger);
19
20     List<Proceso> obtenerCola();
21
22     void limpiarCola();
23 }
24

```

Figura 2: Código fuente de Scheduler.java.

FcfsScheduler.java

Implementa el algoritmo FCFS, donde los procesos se ejecutan en el mismo orden en que llegaron a la cola de planificación.

```

1 package com.gestorprocesos.schedulers;
2
3 import com.gestorprocesos.models.Proceso;
4
5 public class FcfsScheduler extends AbstractScheduler {
6     public FcfsScheduler() {
7         super("FCFS");
8     }
9
10    @Override
11    protected int calcularTiempoEjecucion(Proceso proceso) {
12        return proceso.getTiempoRestante();
13    }
14 }
15

```

Figura 3: Código fuente de FcfsScheduler.java.

SjfScheduler.java

Implementa el algoritmo SJF, seleccionando primero el proceso con menor tiempo restante para reducir el tiempo promedio de espera.

```

1 package com.gestorprocesos.schedulers;
2
3 import java.util.Comparator;
4
5 import com.gestorprocesos.models.Proceso;
6
7 public class SjfScheduler extends AbstractScheduler {
8     public SjfScheduler() {
9         super("SJF");
10    }
11
12    @Override
13    protected int calcularTiempoEjecucion(Proceso proceso) {
14        return proceso.getTiempoRestante();
15    }
16
17    @Override
18    protected Comparator<Proceso> obtenerComparador() {
19        return Comparator.comparingInt(Proceso::getTiempoRestante).thenComparingLong(Proceso::getPid);
20    }
21 }
22

```

Figura 4: Código fuente de SjfScheduler.java.

RoundRobinScheduler.java

Implementa Round Robin, asignando un quantum a cada proceso para simular turnos de CPU y dar mayor equilibrio entre procesos.

```

1 package com.gestorprocesos.schedulers;
2
3 import com.gestorprocesos.models.Proceso;
4
5 public class RoundRobinScheduler extends AbstractScheduler {
6     private int quantum;
7
8     public RoundRobinScheduler(int quantum) {
9         super("Round Robin");
10        this.quantum = Math.max(1, quantum);
11    }
12
13    public void setQuantum(int quantum) {
14        this.quantum = Math.max(1, quantum);
15    }
16
17    public int getQuantum() {
18        return quantum;
19    }
20
21    @Override
22    protected int calcularTiempoEjecucion(Proceso proceso) {
23        return Math.min(quantum, proceso.getTiempoRestante());
24    }
25 }
26

```

Figura 5: Código fuente de RoundRobinScheduler.java.

PriorityScheduler.java

Implementa la planificación por prioridad. Selecciona primero los procesos con mayor prioridad de acuerdo con el valor configurado en cada proceso.

```
1 package com.gestorprocesos.schedulers;
2
3 import java.util.Comparator;
4
5 import com.gestorprocesos.models.Proceso;
6
7 public class PriorityScheduler extends AbstractScheduler {
8     public PriorityScheduler() {
9         super("Prioridades");
10    }
11
12    @Override
13    protected int calcularTiempoEjecucion(Proceso proceso) {
14        return proceso.getTiempoRestante();
15    }
16
17    @Override
18    protected Comparator<Proceso> obtenerComparador() {
19        return Comparator.comparingInt(Proceso::getPrioridad).thenComparingLong(Proceso::getPid);
20    }
21 }
22
```

Figura 6: Código fuente de PriorityScheduler.java.

ResourceManager.java

Administra los recursos del simulador, como memoria, CPU y recursos compartidos. También valida si un proceso puede recibir recursos o debe esperar.

```

package com.gestorprocesos.managers;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

import com.gestorprocesos.models.Proceso;

public class ResourceManager {
    private final int cpuTotal;
    private final int memoriaTotal;
    private int cpuUsada;
    private int memoriaUsada;
    private long pidEnCPU;
    private final Set<String> recursosOcupados;
    private final Map<Long, List<String>> recursosPorProceso;
    private final Map<Long, Integer> memoriaPorProceso;
    private String ultimoMensaje;

    public ResourceManager(int cpuTotal, int memoriaTotal) {
        this.cpuTotal = Math.max(1, cpuTotal);
        this.memoriaTotal = Math.max(1, memoriaTotal);
        this.cpuUsada = 0;
        this.memoriaUsada = 0;
        this.pidEnCPU = -1;
        this.recursosOcupados = new HashSet<>();
        this.recursosPorProceso = new HashMap<>();
        this.memoriaPorProceso = new HashMap<>();
        this.ultimoMensaje = "Sistema inicializado.";
    }

    public boolean solicitarRecursos(Proceso proceso) {
        int memoriaSolicitada = proceso.getMemoriaSolicitada();
        List<String> recursosSolicitados = proceso.getRecursosSolicitados();

        if (memoriaUsada + memoriaSolicitada > memoriaTotal) {
            ultimoMensaje = "No hay memoria suficiente para el proceso PID " + proceso.getPid() + ".";
            return false;
        }
    }

```

Figura 7: Código fuente de ResourceManager.java, parte 1.

```

        for (String recurso : recursosSolicitados) {
            if (recursosOcupados.contains(recurso)) {
                ultimoMensaje = "Conflicto de recursos: el recurso " + recurso + " ya está ocupado.";
                return false;
            }
        }

        memoriaUsada += memoriaSolicitada;
        memoriaPorProceso.put(proceso.getPid(), memoriaSolicitada);
        recursosPorProceso.put(proceso.getPid(), new ArrayList<>(recursosSolicitados));
        recursosOcupados.addAll(recursosSolicitados);
        proceso.asignarRecursos(memoriaSolicitada, recursosSolicitados);
        ultimoMensaje = "Recursos asignados al proceso PID " + proceso.getPid() + ".";
        return true;
    }

    public boolean solicitarCPU(Proceso proceso) {
        if (cpuUsada >= cpuTotal) {
            ultimoMensaje = "CPU ocupada. El proceso PID " + proceso.getPid() + " debe esperar.";
            return false;
        }

        cpuUsada = 1;
        pidEnCPU = proceso.getPid();
        ultimoMensaje = "CPU asignada al proceso PID " + proceso.getPid() + ".";
        return true;
    }

    public void liberarCPU(Proceso proceso) {
        if (pidEnCPU == proceso.getPid()) {
            cpuUsada = 0;
            pidEnCPU = -1;
            ultimoMensaje = "CPU liberada por el proceso PID " + proceso.getPid() + ".";
        }
    }

    public void liberarRecursos(Proceso proceso) {
        Integer memoriaAsignada = memoriaPorProceso.remove(proceso.getPid());
        if (memoriaAsignada != null) {
            memoriaUsada = Math.max(0, memoriaUsada - memoriaAsignada);
        }
    }

```

Figura 8: Código fuente de ResourceManager.java, parte 2.

```

public void liberarRecursos(Proceso proceso) {
    Integer memoriaAsignada = memoriaPorProceso.remove(proceso.getPid());
    if (memoriaAsignada != null) {
        memoriaUsada = Math.max(0, memoriaUsada - memoriaAsignada);
    }

    List<String> recursosAsignados = recursosPorProceso.remove(proceso.getPid());
    if (recursosAsignados != null) {
        recursosOcupados.removeAll(recursosAsignados);
    }

    liberarCPU(proceso);
    proceso.liberarRecursos();
    ultimoMensaje = "Recursos liberados por el proceso PID " + proceso.getPid() + ".";
}

```

Figura 9: Código fuente de ResourceManager.java, parte 3.

ProcessManager.java

Gestiona el ciclo de vida de los procesos: creación, cambios de estado, suspensión, reanudación, terminación e historial. También se relaciona con la mensajería entre procesos.

```

public class ProcessManager {
    public void enviarMensaje(long pidOrigen, long pidDestino, String contenido) {
        messageQueue.enqueue(mensaje);
        procesarMensajesPendientes();
        logger.registrar("MENSAJE", "Mensaje enviado de PID " + pidOrigen + " a PID " + pidDestino + ".");
    }

    public void procesarMensajesPendientes() {
        if (messageQueue.estaVacia()) {
            return;
        }
        for (MensajeProceso mensaje : messageQueue.extraerTodos()) {
            Proceso destino = procesosActivos.get(mensaje.getDestinoPid());
            if (destino != null) {
                destino.recibirMensaje(mensaje);
            }
        }
    }

    public void mostrarProcesosActivos() {
        System.out.println("\n===== PROCESOS ACTIVOS =====");
        if (procesosActivos.isEmpty()) {
            System.out.println("No hay procesos activos.");
            return;
        }
        for (Proceso proceso : procesosActivos.values()) {
            System.out.println(proceso);
            if (!proceso.getMensajesRecibidos().isEmpty()) {
                System.out.println(" Mensajes: " + proceso.getMensajesRecibidos());
            }
        }
    }

    public void mostrarHistorial() {
        System.out.println("\n===== HISTORIAL DE PROCESOS =====");
        if (historial.isEmpty()) {
            System.out.println("Aún no hay procesos terminados.");
            return;
        }
        for (Proceso proceso : historial) {
            System.out.println(proceso);
        }
    }
}

```

Figura 10: Código fuente de ProcessManager.java.

1.8. Explicación del archivo pom.xml

El archivo pom.xml concentra la configuración principal de Maven. En él se definen datos como groupId com.gestorprocesos, artifactId simulador-gestor-procesos, versión 1.0.0, uso de

Java 17 y los plugins `maven-compiler-plugin` y `exec-maven-plugin`.

Con esta configuración se puede compilar el código, generar el archivo JAR, ejecutar la clase principal desde Maven y mantener la compatibilidad con Java 17. No se añadieron dependencias externas, ya que el simulador trabaja con Java estándar y los plugins necesarios para su construcción.

1.9. Instrucciones de compilación y ejecución

Desde la carpeta principal del repositorio se utiliza el siguiente comando:

```
mvn clean package
```

Este comando elimina archivos generados anteriormente, compila el código fuente y empaqueta la aplicación en un archivo JAR.

Para ejecutar la aplicación con Maven:

```
mvn exec:java
```

Para ejecutar la aplicación desde las clases compiladas:

```
java -cp target/classes com.gestorprocesos.main.App
```

Otros comandos básicos de Maven son:

- `mvn clean package`: compilar y empacar.
- `mvn exec:java`: ejecutar la aplicación.
- `mvn clean`: limpiar artefactos compilados.
- `mvn help:effective-pom`: ver información efectiva del proyecto.

1.10. Estructura recomendada del README.md

Un README completo puede incluir el nombre del sistema, una descripción general, sus características, arquitectura, algoritmos de planificación, requisitos técnicos, instrucciones de compilación y ejecución, menú principal, ejemplos de uso y notas finales.

2. Introducción

Este documento explica el diseño y funcionamiento del *Simulador de Gestor de Procesos para Sistemas Operativos*. El sistema se desarrolló en Java con Maven y utiliza una interfaz de consola para representar temas como administración de procesos, planificación de CPU, memoria, recursos compartidos, sincronización y comunicación entre procesos.

El simulador no intenta reemplazar ni copiar el funcionamiento completo de un kernel real. Su finalidad es mostrar, de forma sencilla, cómo se comportan algunas operaciones básicas de un sistema operativo al administrar procesos y recursos.

3. Objetivo general

Desarrollar un simulador de gestor de procesos en Java que permita representar, mediante consola, la creación, planificación, ejecución, suspensión, reanudación, terminación y administración de procesos, recursos y sincronización en un sistema operativo simulado.

4. Objetivos específicos

- Implementar una PCB que represente cada proceso.
- Simular estados de procesos y sus transiciones.
- Administrar memoria, CPU y recursos compartidos.
- Integrar algoritmos de planificación FCFS, SJF, Round Robin y Prioridades.
- Simular comunicación entre procesos mediante mensajes.
- Simular sincronización con productor-consumidor usando semáforos.
- Registrar eventos en logs.
- Calcular estadísticas de ejecución.
- Proveer una interfaz CLI para ejecutar pruebas manuales.

5. Planteamiento del problema

En un sistema operativo, la administración de procesos y recursos determina el rendimiento, la equidad de CPU y la estabilidad general del sistema. Cuando varios procesos compiten por memoria, CPU y recursos compartidos, el sistema debe decidir quién ejecuta, quién espera y qué ocurre cuando faltan recursos.

El problema abordado consiste en representar estas decisiones de forma clara y controlada desde consola, de manera que sea posible observar el comportamiento de distintos algoritmos de planificación y mecanismos de sincronización.

6. Justificación

El simulador facilita el aprendizaje porque muestra en ejecución varios conceptos de sistemas operativos. En lugar de quedarse sólo en la teoría, permite ver cambios de estado, asignación de memoria, uso de CPU, liberación de recursos y diferencias entre algoritmos de planificación.

Además, el uso de Java y Maven facilita el mantenimiento, la portabilidad y la organización modular del código.

7. Descripción general del sistema

El sistema está compuesto por una clase principal que presenta un menú de consola y por varios módulos especializados: creación y administración de procesos, asignación y liberación de recursos, planificación por algoritmo, mensajería entre procesos, simulación de productor-consumidor, registro de eventos y estadísticas.

La aplicación crea procesos manualmente o mediante datos de prueba aleatorios y los envía a una cola de planificación. Dependiendo del algoritmo seleccionado, el scheduler decide el orden de ejecución. Los procesos que terminan liberan sus recursos y pasan al historial.

8. Arquitectura del sistema

La arquitectura es modular y orientada a objetos. Cada paquete cumple una responsabilidad definida.

- **Capa de presentación:** `App` y `ConsoleInput`. Manejan la interacción con el usuario.
- **Capa de dominio:** `Proceso`, `MensajeProceso` y enums. Representan las entidades del sistema.
- **Capa de negocio:** `ProcessManager` y `ResourceManager`. Encargados de la lógica principal.
- **Capa de planificación:** `Scheduler` y sus implementaciones. Deciden el orden de ejecución.
- **Capa de soporte:** `Logger`, `StatisticsManager`, `TimeSimulator` y `RandomDataGenerator`.
- **Capa de sincronización:** `ProducerConsumerSimulation` y `MessageQueue`.

9. Explicación de módulos

9.1. main

Contiene App, que coordina el menú principal y conecta todos los módulos.

9.2. models

Contiene las estructuras de datos centrales del simulador.

9.3. managers

Contiene el manejo del estado global de procesos y recursos.

9.4. schedulers

Contiene la lógica de algoritmos de planificación.

9.5. synchronization

Contiene la simulación de comunicación y sincronización.

9.6. utils

Contiene utilidades comunes de la aplicación.

10. Gestión de procesos

Cada proceso se representa mediante la clase **Proceso**, que funciona como una PCB simulada. Esta entidad almacena PID único, nombre del proceso, estado, prioridad, tiempo de ejecución original, tiempo restante, memoria solicitada y asignada, recursos solicitados y asignados, motivo de terminación y mensajes recibidos.

La clase **ProcessManager** crea procesos, los mantiene en una estructura de procesos activos y conserva un historial de terminados.

11. Estados de procesos

Los estados definidos en el sistema son:

Estado	Significado
LISTO	El proceso puede entrar a CPU.
EJECUTANDO	El proceso está usando CPU.
ESPERANDO	El proceso espera memoria o recursos.
SUSPENDIDO	El proceso fue detenido manualmente.
TERMINADO	El proceso finalizó y liberó recursos.

Las transiciones principales son: de LISTO a EJECUTANDO cuando el scheduler lo selecciona; de EJECUTANDO a TERMINADO si finaliza su tiempo; de LISTO a SUSPENDIDO por acción manual; de SUSPENDIDO a LISTO al reanudar; y de ESPERANDO a LISTO cuando los recursos vuelven a estar disponibles.

12. Planificación de CPU

La planificación decide qué proceso entra a CPU. En el sistema esto se implementa con una interfaz común **Scheduler** y una clase base **AbstractScheduler**. La cola de planificación contiene sólo procesos en estado LISTO. El scheduler selecciona el siguiente proceso, solicita CPU, lo ejecuta durante una cantidad de tiempo simulada, reduce su tiempo restante y luego libera la CPU.

13. Explicación de algoritmos

13.1. FCFS

First-Come, First-Served. Ejecuta los procesos en orden de llegada, no expulsa al proceso una vez que entra a CPU, es simple y predecible, aunque puede causar largas esperas si el primer proceso es muy grande.

13.2. SJF

Shortest Job First. Selecciona el proceso con menor tiempo restante, reduce el tiempo promedio de espera y favorece trabajos cortos, aunque puede causar inanición de procesos largos.

13.3. Round Robin

Cada proceso recibe un quantum fijo. Si no termina, regresa al final de la cola. Mejora la equidad, es ideal para simular multitarea con turnos y el quantum se configura antes de ejecutar el algoritmo.

13.4. Prioridades

Selecciona primero el proceso con mayor prioridad. En esta implementación, los valores numéricos más bajos indican mayor prioridad. Este método puede dejar esperando a procesos de prioridad baja, pero resulta útil para representar tareas críticas o de sistema.

14. Administración de recursos

La clase `ResourceManager` controla CPU disponible, memoria total, memoria usada y recursos ocupados. Cuando un proceso se crea, el sistema verifica si existe memoria suficiente y si los recursos solicitados están libres. Si la validación es positiva, los recursos se asignan. En caso contrario, el proceso queda esperando.

Los recursos manejados por la simulación incluyen CPU, memoria y recursos lógicos como DISCO, RED, GPU, IMPRESORA u otros definidos en el simulador.

15. Comunicación entre procesos

La comunicación entre procesos se simula mediante la clase `MensajeProceso` y una cola interna `MessageQueue`. El flujo consiste en que el proceso origen envía un mensaje, el mensaje se encola, el sistema procesa la cola, y el proceso destino recibe el mensaje y lo guarda en su lista de mensajes recibidos.

Esto permite observar comunicación lógica entre procesos sin implementar un IPC real del sistema operativo.

16. Sincronización

La sincronización se aborda de dos maneras: coordinación de acceso a recursos mediante el `ResourceManager` y simulación de exclusión mutua y control de buffer en productor-consumidor usando semáforos de Java. La exclusión mutua evita condiciones de carrera en la simulación concurrente.

17. Problema productor-consumidor

El módulo `ProducerConsumerSimulation` representa un buffer acotado con un semáforo para espacios disponibles, un semáforo para elementos disponibles y un semáforo mutex para exclusión mutua.

El productor produce elementos y los coloca en el buffer. Si el buffer está lleno, el productor espera. El consumidor retira elementos; si el buffer está vacío, el consumidor espera. Este módulo demuestra

de forma práctica bloqueo, espera y liberación de recursos.

18. Manejo de memoria

La memoria se asigna por proceso al momento de la creación si existe capacidad disponible. Los criterios de validación son: si `memoriaUsada` + `memoriaSolicitada` supera `memoriaTotal`, se rechaza la asignación; si la asignación es válida, la memoria se marca como ocupada; al terminar el proceso, la memoria se libera.

Esto permite observar el efecto de la presión de memoria y de los procesos en espera.

19. Logs del sistema

La clase `Logger` registra los eventos del simulador en orden cronológico. Entre los eventos registrados se incluyen creación de procesos, cambios de estado, asignación y liberación de recursos, ejecución de procesos, terminaciones, mensajes entre procesos y eventos de sincronización.

Los logs ayudan a verificar lo que ocurre internamente en cada acción del usuario.

20. Estadísticas del sistema

`StatisticsManager` calcula cantidad de procesos ejecutados, tiempo promedio de ejecución, memoria usada actual y CPU usada actual. Estas estadísticas permiten evaluar el comportamiento general del simulador después de varias ejecuciones.

21. Flujo de ejecución

1. El usuario inicia la aplicación.
2. Se genera un conjunto de procesos de prueba o se crean manualmente.
3. El `ResourceManager` evalúa memoria y recursos.
4. El usuario selecciona un algoritmo de planificación.
5. El scheduler ejecuta los procesos listos.
6. Los procesos pueden suspenderse, reanudarse o terminarse.
7. Se liberan recursos al finalizar.
8. El sistema registra logs y estadísticas.

22. Casos de uso

Caso de uso	Entrada	Resultado
Crear proceso	Nombre, prioridad, tiempo de ejecución, memoria y recursos.	Proceso creado en estado LISTO o ESPERANDO.
Ejecutar scheduler	Selección del algoritmo.	Los procesos se ejecutan según la política elegida.
Terminar proceso	PID y motivo.	El proceso libera sus recursos y pasa al historial.
Enviar mensaje	PID origen, PID destino y contenido.	El mensaje se entrega al proceso destino.
Simular productor-consumidor	Comando del menú.	Se observan esperas, producción y consumo de elementos.

23. Ejemplos de ejecución

- **Ejemplo A: creación de procesos.** Opción 1 del menú. El sistema genera procesos aleatorios. Algunos pueden quedar en ESPERANDO si falta memoria o hay conflicto de recursos.
- **Ejemplo B: FCFS.** Opción 10, seleccionar FCFS. Opción 13 para ver la cola. Opción 4 para ejecutar. Se observa el orden de llegada.
- **Ejemplo C: Round Robin.** Opción 10, seleccionar Round Robin e ingresar quantum. Opción 4. Cada proceso recibe un turno limitado y vuelve a la cola si no termina.
- **Ejemplo D: comunicación.** Opción 11. Se elige un PID origen y un PID destino. El mensaje queda asociado al proceso destino.
- **Ejemplo E: productor-consumidor.** Opción 12. Se observan logs de producción, consumo, espera y liberación de recursos.

24. Pruebas del sistema

En esta sección se muestran las pruebas realizadas sobre el simulador. Las capturas incluidas sirven como evidencia de compilación, ejecución, planificación, comunicación, sincronización, logs y estadísticas.

24.1. Prueba 1: compilación del proyecto

Comando utilizado:

```
mvn clean package
```

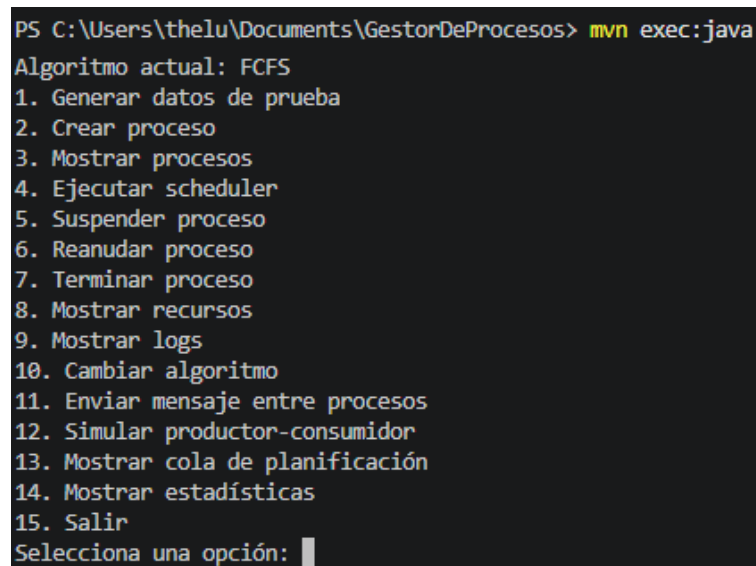
Resultado esperado: la compilación debe finalizar sin errores y los archivos generados deben aparecer dentro de la carpeta **target**.

24.2. Prueba 2: ejecución del menú principal

Comando utilizado:

```
mvn exec:java
```

Resultado esperado: la consola debe mostrar el menú principal del simulador y permitir seleccionar opciones.

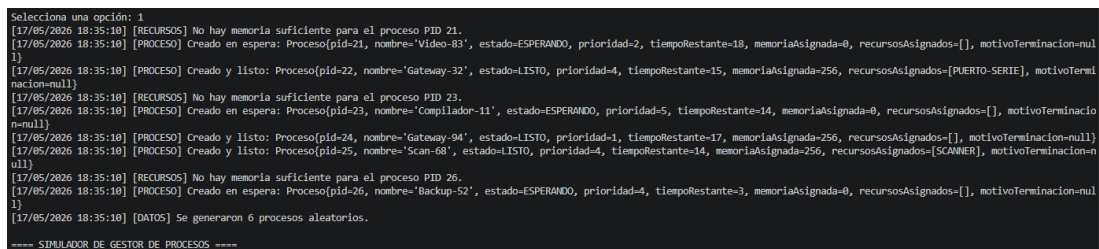


```
PS C:\Users\theLu\Documents\GestorDeProcesos> mvn exec:java
Algoritmo actual: FCFS
1. Generar datos de prueba
2. Crear proceso
3. Mostrar procesos
4. Ejecutar scheduler
5. Suspender proceso
6. Reanudar proceso
7. Terminar proceso
8. Mostrar recursos
9. Mostrar logs
10. Cambiar algoritmo
11. Enviar mensaje entre procesos
12. Simular productor-consumidor
13. Mostrar cola de planificación
14. Mostrar estadísticas
15. Salir
Selecciona una opción: |
```

Figura 11: Ejecución del menú principal del simulador en consola.

24.3. Prueba 3: creación de procesos

Resultado esperado: se deben crear procesos con PID único, estado inicial coherente y asignación de memoria o recursos según disponibilidad.



```
Selecciona una opción: 1
[17/05/2026 18:35:10] [RECURSOS] No hay memoria suficiente para el proceso PID 21.
[17/05/2026 18:35:10] [PROCESO] Creado en espera: Proceso[pid=21, nombre='Video-83', estado=ESPERANDO, prioridad=2, tiempoRestante=18, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null]
[17/05/2026 18:35:10] [PROCESO] Creado y listo: Proceso[pid=22, nombre='Gateway-32', estado=LISTO, prioridad=4, tiempoRestante=15, memoriaAsignada=256, recursosAsignados=[PUERTO-SERIE], motivoTerminacion=null]
[17/05/2026 18:35:10] [RECURSOS] No hay memoria suficiente para el proceso PID 23.
[17/05/2026 18:35:10] [PROCESO] Creado en espera: Proceso[pid=23, nombre='Compilador-11', estado=ESPERANDO, prioridad=5, tiempoRestante=14, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null]
[17/05/2026 18:35:10] [PROCESO] Creado y listo: Proceso[pid=24, nombre='Gateway-94', estado=LISTO, prioridad=1, tiempoRestante=17, memoriaAsignada=256, recursosAsignados=[], motivoTerminacion=null]
[17/05/2026 18:35:10] [PROCESO] Creado y listo: Proceso[pid=25, nombre='Scan-68', estado=LISTO, prioridad=4, tiempoRestante=14, memoriaAsignada=256, recursosAsignados=[SCANNER], motivoTerminacion=null]
[17/05/2026 18:35:10] [RECURSOS] No hay memoria suficiente para el proceso PID 26.
[17/05/2026 18:35:10] [PROCESO] Creado en espera: Proceso[pid=26, nombre='Backup-52', estado=ESPERANDO, prioridad=4, tiempoRestante=3, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null]
[17/05/2026 18:35:10] [DATOS] Se generaron 6 procesos aleatorios.
==== SIMULADOR DE GESTOR DE PROCESOS =====
```

Figura 12: Creación de procesos en el simulador.

24.4. Prueba 4: planificación de CPU

Resultado esperado: el algoritmo seleccionado debe ordenar y ejecutar los procesos conforme a su política, ya sea FCFS, SJF, Round Robin o Prioridades.

```
15. Salir          4
[17/05/2026 18:45:38] [SCHEDULER] Iniciando planificador FCFS.
[17/05/2026 18:45:38] [EJECUCIÓN] Proceso PID 6 ejecutándose con FCFS.
[17/05/2026 18:45:39] [TERMINACION] Proceso PID 6 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:45:39] [EJECUCIÓN] Proceso PID 7 ejecutándose con FCFS.
[17/05/2026 18:45:39] [TERMINACION] Proceso PID 7 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:45:39] [EJECUCIÓN] Proceso PID 8 ejecutándose con FCFS.
[17/05/2026 18:45:39] [TERMINACION] Proceso PID 8 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:45:39] [EJECUCIÓN] Proceso PID 17 ejecutándose con FCFS.
[17/05/2026 18:45:39] [TERMINACION] Proceso PID 17 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:45:39] [EJECUCIÓN] Proceso PID 22 ejecutándose con FCFS.
[17/05/2026 18:45:39] [TERMINACION] Proceso PID 22 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:45:39] [EJECUCIÓN] Proceso PID 24 ejecutándose con FCFS.
[17/05/2026 18:45:39] [TERMINACION] Proceso PID 24 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:45:39] [EJECUCIÓN] Proceso PID 25 ejecutándose con FCFS.
[17/05/2026 18:45:39] [TERMINACION] Proceso PID 25 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:45:39] [SCHEDULER] Finalizó el planificador FCFS.
[17/05/2026 18:45:39] [RECURSOS] Proceso PID 9 pasó a LISTO.
[17/05/2026 18:45:39] [RECURSOS] Proceso PID 10 pasó a LISTO.
[17/05/2026 18:45:39] [RECURSOS] Proceso PID 13 pasó a LISTO.
```

Figura 13: Ejecución del algoritmo FCFS.

```
Selecciona una opción: 4
[17/05/2026 18:46:43] [SCHEDULER] Iniciando planificador SJF.
[17/05/2026 18:46:43] [EJECUCIÓN] Proceso PID 9 ejecutándose con SJF.
[17/05/2026 18:46:43] [TERMINACION] Proceso PID 9 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:46:43] [EJECUCIÓN] Proceso PID 13 ejecutándose con SJF.
[17/05/2026 18:46:43] [TERMINACION] Proceso PID 13 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:46:43] [EJECUCIÓN] Proceso PID 10 ejecutándose con SJF.
[17/05/2026 18:46:44] [TERMINACION] Proceso PID 10 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:46:44] [SCHEDULER] Finalizó el planificador SJF.
[17/05/2026 18:46:44] [RECURSOS] Proceso PID 11 pasó a LISTO.
[17/05/2026 18:46:44] [RECURSOS] Proceso PID 12 pasó a LISTO.
[17/05/2026 18:46:44] [RECURSOS] Proceso PID 20 pasó a LISTO.
[17/05/2026 18:46:44] [RECURSOS] Proceso PID 26 pasó a LISTO.
[17/05/2026 18:46:44] [RECURSOS] Proceso PID 31 pasó a LISTO.
```

Figura 14: Ejecución del algoritmo SJF.


```

Selecciona una opción: 4
[17/05/2026 18:48:29] [SCHEDULER] Iniciando planificador Prioridades.
[17/05/2026 18:48:29] [EJECUCIÓN] Proceso PID 14 ejecutándose con Prioridades.
[17/05/2026 18:48:29] [TERMINACION] Proceso PID 14 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:48:29] [EJECUCIÓN] Proceso PID 16 ejecutándose con Prioridades.
[17/05/2026 18:48:29] [TERMINACION] Proceso PID 16 finalizado por FINALIZACION_CORRECTA.
[17/05/2026 18:48:29] [SCHEDULER] Finalizó el planificador Prioridades.
[17/05/2026 18:48:29] [RECURSOS] Proceso PID 15 pasó a LISTO.
[17/05/2026 18:48:29] [RECURSOS] Proceso PID 19 pasó a LISTO.
[17/05/2026 18:48:29] [RECURSOS] Proceso PID 36 pasó a LISTO.

```

Figura 16: Ejecución del algoritmo por prioridades.

24.5. Prueba 5: comunicación y sincronización

Resultado esperado: los mensajes deben entregarse al proceso destino y la simulación productor-consumidor debe mostrar eventos de producción, consumo, espera y liberación.

```

==== SIMULADOR DE GESTOR DE PROCESOS ====
Algoritmo actual: FCFS
1. Generar datos de prueba
2. Crear proceso
3. Mostrar procesos
4. Ejecutar scheduler
5. Suspender proceso
6. Reanudar proceso
7. Terminar proceso
8. Mostrar recursos
9. Mostrar logs
10. Cambiar algoritmo
11. Enviar mensaje entre procesos
12. Simular productor-consumidor
13. Mostrar cola de planificación
14. Mostrar estadísticas
15. Salir
Selecciona una opción: 11
PID origen: 8
PID destino: 9
Mensaje: hola
[17/05/2026 18:22:26] [MENSAJE] Mensaje enviado de PID 8 a PID 9.

```

Figura 17: Comunicación entre procesos mediante mensajes.

```

11. Enviar mensaje entre procesos
12. Simular productor-consumidor
13. Mostrar cola de planificación
14. Mostrar estadísticas
15. Salir
Selecciona una opción: 12
[17/05/2026 18:24:29] [SINCRONIZACIÓN] Iniciando simulación productor-consumidor.
[17/05/2026 18:24:29] [SINCRONIZACIÓN] Consumidor en espera: buffer vacío.
[17/05/2026 18:24:29] [PRODUCTOR] Produjo Dato-1. Buffer: [Dato-1]
[17/05/2026 18:24:29] [CONSUMIDOR] Consumió Dato-1. Buffer: []
[17/05/2026 18:24:29] [PRODUCTOR] Produjo Dato-2. Buffer: [Dato-2]
[17/05/2026 18:24:29] [CONSUMIDOR] Consumió Dato-2. Buffer: []
[17/05/2026 18:24:29] [PRODUCTOR] Produjo Dato-3. Buffer: [Dato-3]
[17/05/2026 18:24:29] [PRODUCTOR] Produjo Dato-4. Buffer: [Dato-3, Dato-4]
[17/05/2026 18:24:29] [CONSUMIDOR] Consumió Dato-3. Buffer: [Dato-4]
[17/05/2026 18:24:29] [PRODUCTOR] Produjo Dato-5. Buffer: [Dato-4, Dato-5]
[17/05/2026 18:24:29] [PRODUCTOR] Produjo Dato-6. Buffer: [Dato-4, Dato-5, Dato-6]
[17/05/2026 18:24:29] [CONSUMIDOR] Consumió Dato-4. Buffer: [Dato-5, Dato-6]
[17/05/2026 18:24:29] [CONSUMIDOR] Consumió Dato-5. Buffer: [Dato-6]
[17/05/2026 18:24:29] [CONSUMIDOR] Consumió Dato-6. Buffer: []
[17/05/2026 18:24:29] [SINCRONIZACIÓN] Finalizó la simulación productor-consumidor.

```

Figura 18: Simulación del problema productor-consumidor.

24.6. Prueba 6: logs y estadísticas

Resultado esperado: el sistema debe presentar registros cronológicos y estadísticas acumuladas después de ejecutar procesos.

```

===== LOGS DEL SISTEMA =====
1. [17/05/2026 18:09:41] [PROCESO] Creado y listo: Proceso{pid=1, nombre='Compilador-47', estado=LISTO, prioridad=1, tiempoRestante=13, memoriaAsignada=1280, recursosAsignados=[], motivoTerminacion=null}
2. [17/05/2026 18:09:41] [PROCESO] Creado y listo: Proceso{pid=2, nombre='Email-19', estado=LISTO, prioridad=3, tiempoRestante=8, memoriaAsignada=1536, recursosAsignados=[PUERTO-SERIE], motivoTerminacion=null}
3. [17/05/2026 18:09:41] [PROCESO] Creado y listo: Proceso{pid=3, nombre='Video-1', estado=LISTO, prioridad=3, tiempoRestante=9, memoriaAsignada=768, recursosAsignados=[DISCO-1], motivoTerminacion=null}
4. [17/05/2026 18:09:41] [RECURSOS] No hay memoria suficiente para el proceso PID 4.
5. [17/05/2026 18:09:41] [PROCESO] Creado en espera: Proceso{pid=4, nombre='Copia-52', estado=ESPERANDO, prioridad=4, tiempoRestante=15, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null}
6. [17/05/2026 18:09:41] [PROCESO] Creado y listo: Proceso{pid=5, nombre='Servidor-38', estado=LISTO, prioridad=1, tiempoRestante=3, memoriaAsignada=256, recursosAsignados=[], motivoTerminacion=null}
7. [17/05/2026 18:09:41] [DATOS] Se generaron 5 procesos aleatorios.
8. [17/05/2026 18:12:13] [RECURSOS] No hay memoria suficiente para el proceso PID 6.
9. [17/05/2026 18:12:13] [PROCESO] Creado en espera: Proceso{pid=6, nombre='Calculadora-12', estado=ESPERANDO, prioridad=4, tiempoRestante=5, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null}
10. [17/05/2026 18:12:13] [RECURSOS] No hay memoria suficiente para el proceso PID 7.
11. [17/05/2026 18:12:13] [PROCESO] Creado en espera: Proceso{pid=7, nombre='Análisis-47', estado=ESPERANDO, prioridad=5, tiempoRestante=15, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null}
12. [17/05/2026 18:12:13] [RECURSOS] No hay memoria suficiente para el proceso PID 8.
13. [17/05/2026 18:12:13] [PROCESO] Creado en espera: Proceso{pid=8, nombre='Gateway-21', estado=ESPERANDO, prioridad=4, tiempoRestante=13, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null}
14. [17/05/2026 18:12:13] [RECURSOS] No hay memoria suficiente para el proceso PID 9.
15. [17/05/2026 18:12:13] [PROCESO] Creado en espera: Proceso{pid=9, nombre='Editor-37', estado=ESPERANDO, prioridad=4, tiempoRestante=17, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null}
16. [17/05/2026 18:12:13] [DATOS] Se generaron 4 procesos aleatorios.
17. [17/05/2026 18:14:55] [SCHEDULER] Iniciando planificador FCFS.
18. [17/05/2026 18:14:55] [EJECUCIÓN] Proceso PID 1 ejecutándose con FCFS.
19. [17/05/2026 18:14:55] [TERMINACION] Proceso PID 1 finalizado por FINALIZACION_CORRECTA.
20. [17/05/2026 18:14:55] [EJECUCIÓN] Proceso PID 2 ejecutándose con FCFS.
21. [17/05/2026 18:14:55] [TERMINACION] Proceso PID 2 finalizado por FINALIZACION_CORRECTA.
22. [17/05/2026 18:14:55] [EJECUCIÓN] Proceso PID 3 ejecutándose con FCFS.
23. [17/05/2026 18:14:56] [TERMINACION] Proceso PID 3 finalizado por FINALIZACION_CORRECTA.
24. [17/05/2026 18:14:56] [EJECUCIÓN] Proceso PID 5 ejecutándose con FCFS.
25. [17/05/2026 18:14:56] [TERMINACION] Proceso PID 5 finalizado por FINALIZACION_CORRECTA.
26. [17/05/2026 18:14:56] [SCHEDULER] Finalizó el planificador FCFS.
27. [17/05/2026 18:14:56] [RECURSOS] Proceso PID 4 pasó a LISTO.
28. [17/05/2026 18:14:56] [RECURSOS] Proceso PID 6 pasó a LISTO.
29. [17/05/2026 18:14:56] [RECURSOS] Proceso PID 7 pasó a LISTO.
30. [17/05/2026 18:15:51] [ERROR] No se pudo enviar el mensaje. Verifica los PIDs.
31. [17/05/2026 18:15:55] [RECURSOS] No hay memoria suficiente para el proceso PID 10.
32. [17/05/2026 18:15:55] [PROCESO] Creado en espera: Proceso{pid=10, nombre='Descarga-46', estado=ESPERANDO, prioridad=4, tiempoRestante=19, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null}
33. [17/05/2026 18:15:55] [RECURSOS] No hay memoria suficiente para el proceso PID 11.
34. [17/05/2026 18:15:55] [PROCESO] Creado en espera: Proceso{pid=11, nombre='Chat-44', estado=ESPERANDO, prioridad=3, tiempoRestante=3, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null}
35. [17/05/2026 18:15:55] [RECURSOS] No hay memoria suficiente para el proceso PID 12.
36. [17/05/2026 18:15:55] [PROCESO] Creado en espera: Proceso{pid=12, nombre='Servidor-60', estado=ESPERANDO, prioridad=4, tiempoRestante=16, memoriaAsignada=0, recursosAsignados=[], motivoTerminacion=null}

```

Figura 19: Logs del sistema y estadísticas acumuladas.

25. Resultados esperados

Al ejecutar correctamente el simulador, se espera observar creación de procesos con PID único, cambios de estado coherentes, planificación según el algoritmo elegido, asignación y liberación de recursos, procesos en espera cuando faltan recursos, mensajes entregados entre procesos, sincronización visible en productor-consumidor, logs ordenados y estadísticas acumuladas.

26. Conclusiones

El desarrollo del simulador ayudó a entender de manera práctica cómo se administran procesos, memoria, recursos y algoritmos de planificación. La ejecución en consola hizo más claro observar los cambios de estado, el orden de ejecución y la importancia de sincronizar correctamente los procesos.

Conclusión de Narez Roses Erick Eduardo

Al trabajar en el simulador comprendí mejor cómo se administran los procesos dentro de un sistema operativo. También vi que separar el código por módulos ayuda mucho a mantenerlo ordenado y a explicar mejor cada parte.

Conclusión de Jimenez Aguilar Christian Jesus

Este trabajo me ayudó a entender de forma más clara los algoritmos de planificación como FCFS, SJF, Round Robin y Prioridades. Al verlos ejecutarse en consola, pude comparar cómo cambia el orden de los procesos según la estrategia seleccionada.

Conclusión de Llamas Sanchez Adrian

Con el simulador aprendí más sobre asignación de memoria, recursos compartidos y estados de los procesos. También reforcé el uso de Java y Maven para construir una aplicación ordenada y fácil de ejecutar.

Conclusión de Meza Roman Juan Pablo

El simulador me permitió entender mejor la comunicación entre procesos y la sincronización usando el ejemplo de productor-consumidor. Me pareció útil porque muestra conceptos teóricos de sistemas operativos de una manera más visual y práctica.

27. Referencias

- Silberschatz, A., Galvin, P. B. y Gagne, G. *Operating System Concepts*.
- Tanenbaum, A. S. y Bos, H. *Modern Operating Systems*.
- Documentación oficial de Java.
- Documentación oficial de Maven.
- Apuntes de clase sobre procesos, planificación y sincronización.

A. Resumen de clases reales del proyecto

Clases y archivos principales incluidos en el sistema:

- App
- Proceso
- MensajeProceso
- EstadoProceso
- MotivoTerminacion
- SchedulerType
- ProcessManager
- ResourceManager
- Scheduler
- AbstractScheduler
- FcfsScheduler
- SjfScheduler
- RoundRobinScheduler
- PriorityScheduler
- SchedulerFactory
- MessageQueue
- ProducerConsumerSimulation
- Logger
- ConsoleInput
- RandomDataGenerator
- StatisticsManager
- TimeSimulator